WORKSHOP EPSI BAC+5 · DOSSIER D'INGÉNIERIE & DE GOUVERNANCE

PROJECT RESILIENCE-ALPHA

Cyber-résilience et reprise autonome d'une plateforme de paiement

> **Démonstrateur :** un SI de paiement fictif conteneurisé qui subit une attaque contrôlée, la détecte par machine learning et se rétablit **sans intervention humaine**, avec preuve par transaction.
>
> Dépôt : github.com/bigwihodari/Resilience-Alpha · Environnement et données entièrement fictifs.

Consortium : Workshop2026-M2-G<n> · Équipe : [À COMPLÉTER] · Octobre 2026

1. Résumé exécutif

Resilience-Alpha démontre une **chaîne de cyber-résilience de bout en bout** : une plateforme de paiement fictive, une attaque réelle lancée depuis un poste distinct, une détection par apprentissage automatique, une remédiation automatique et une vérification du rétablissement par une nouvelle transaction réussie.

~15 s détection → reprise (RTO)	**100 %** reprise sans humain	**15/15 · 13/13** recettes résilience · failover	**2 serveurs** bascule automatique (HA)

La solution couvre les cinq domaines exigés : développement (microservices de paiement), infrastructure (conteneurisation + observabilité), IA/Data (détection ML + assistant RAG), cybersécurité (attaque, WAF, SOC, secrets, durcissement) et gouvernance (EBIOS RM, PCA/PRA, NIS2/DORA).

Les deux incidents et leurs réponses (distinctes)

Incident	Atteinte	Réponse automatique	Preuve
Déni de service (DoS)	Disponibilité	Bouclier anti-DoS (load-shedding) + relâche auto	Transaction canari réussie
Injection SQL	Compromission	Blocage WAF + contrôle d'intégrité des données	Intégrité confirmée

2. Architecture & flux

Deux postes sur le réseau Wi-Fi du groupe (`RESILIENCE-G<n>`, 192.168.137.x) : le **PC Hôte** exécute la plateforme, le **PC Attaquant** lance les attaques depuis une application dédiée.

Composants (conteneurs Docker Compose)

Service	Rôle	Technologie
frontend	App unique de l'hôte : paiement + cockpit (reverse-proxy)	React + Tailwind, nginx
api	Transactions fictives, /metrics, WAF applicatif	FastAPI
auth	Authentification JWT, bouclier anti-DoS, WAF	FastAPI
db	Comptes & transactions (non exposée)	PostgreSQL
detector	Détection d'anomalies ML (score)	scikit-learn
orchestrator	Moteur de remédiation (décide/agit/vérifie)	FastAPI
prometheus / grafana / loki / promtail / cadvisor	Observabilité (métriques, logs, CPU/mém)	Stack Grafana
traffic-gen	Trafic légitime (baseline du modèle)	Python
assistant	Assistant RAG opérateur (lecture doc, sans privilège)	Ollama + LangChain
attacker (poste distinct)	Console d'attaque tout-en-un (DoS, SQLi)	FastAPI + Chart.js

Flux de résilience

`trafic + attaque → métriques Prometheus + logs Loki → détecteur ML (score) → orchestrateur (décision + action) → vérification canari → cockpit`

Montage haute disponibilité (5 PC)

Deux serveurs (**principal + secours**), deux clients, un attaquant. L'application **client est séparée du serveur** : les clients se paient en direct (transfert entre comptes + annuaire de recherche), le serveur gère transactions, connexions et supervision (cockpit). Le backup **réplique** le principal (comptes + clés d'idempotence).

3. Détection d'anomalies (machine learning)

Données & préparation

Le détecteur interroge Prometheus toutes les 5 s et construit un vecteur de features de débit : requêtes totales/s, requêtes auth/s, taux d'erreurs 4xx/s, 5xx/s. Il apprend la « vie normale » grâce au générateur de trafic légitime.

Modèle

Un **Isolation Forest** est entraîné sur la normale ; en complément, on apprend la distribution normale (moyenne/écart-type) et on mesure l'écart courant par une **distance standardisée** (Mahalanobis diagonale) avec un plancher de bruit adapté au débit. Le flag d'anomalie est piloté par cette distance (fiable sur les gros écarts), l'Isolation Forest servant de corroboration. Aucun seuil codé en dur sur les métriques brutes.

Résultats mesurés

Situation	Débit auth	Latence p95 auth	Score d'anomalie
Normale	~1 req/s	~95 ms	~0
Sous DoS	~35 req/s	~490 ms	100 (anomalie)

Limites

Détection sensible à la qualité de la baseline ; attaques « low-and-slow » moins visibles ; modèle persisté mais à ré-entraîner si le profil de trafic change (endpoint `/retrain`).

4. Remédiation autonome (zero-touch)

L'orchestrateur surveille le détecteur. Sur anomalie confirmée, il **décide**, **agit**, puis **vérifie** — sans intervention humaine.

Playbook disponibilité (DoS)

- Activation du **bouclier** sur l'auth : au-delà d'un débit, les requêtes externes reçoivent un 429 immédiat (sans calcul bcrypt coûteux) → l'auth récupère.
- Vérification par **transaction canari** (login + paiement) → service rétabli.
- Relâche automatique du bouclier quand l'attaque cesse.

Playbook compromission (injection SQL)

- Le WAF bloque la charge utile (403) ; l'orchestrateur lance un **contrôle d'intégrité** des données (comptes attendus, pas d'utilisateur suspect, soldes cohérents).
- Restauration depuis une sauvegarde saine en cas d'altération (RPO = dernier snapshot).

Accès maîtrisés

L'orchestrateur n'a **pas** d'accès god-mode aux conteneurs : il agit via une API d'administration **scoped** (activer le bouclier, contrôler l'intégrité), protégée par un token dédié — un point d'attention explicite du cahier des charges.

Haute disponibilité : bascule & reprise de transaction

Si le serveur principal tombe, l'application client **bascule automatiquement** sur le serveur de secours (upstream nginx). Chaque paiement porte une **clé d'idempotence** : une transaction en cours est **reprise exactement une fois** sur le backup, sans double débit. Le backup se **promeut** autoritaire après quelques secondes d'absence du principal (évite l'écrasement de ses propres écritures à son retour). Deux niveaux combinés : auto-réparation (remédiation) si le principal peut se soigner, bascule si l'hôte crashe.

5. Sécurité (défense)

Dispositif	Rôle
WAF applicatif (auth + api)	Inspection des entrées (signatures OWASP SQLi/XSS) → 403 + métrique + log
Fonction SOC	Loki + Prometheus + vue SOC du cockpit (blocages WAF, état, intégrité)
Moindre privilège	Orchestrateur via API admin scoped ; clients internes via token dédié
Durcissement	`no-new-privileges` sur les services ; DB non exposée ; cAdvisor sans port
Secrets	Chiffrés via SOPS + age ; aucun secret en clair dans le dépôt

Détail : voir `SECURITE.md` dans le dépôt.

6. Résultats des essais

Une recette automatisée (`scripts/recette.sh`) rejoue la chaîne complète et produit un rapport. Dernier passage : **15 OK / 0 KO**.

Vérification	Résultat
Santé des 7 services clés	**OK**
Modèle ML entraîné	**OK**
Transaction nominale	**OK**
DoS → reprise autonome (RECOVERED) + bouclier	**OK**
Transaction réussie pendant l'attaque	**OK**
Injection SQL → contenue par le WAF	**OK**
Contrôle d'intégrité des données	**OK**
Transaction finale après incidents	**OK**

Chaos engineering (`scripts/chaos.sh`) : arrêt brutal d'un conteneur → reprise automatique vérifiée par canari (politique de redémarrage).

Recette haute disponibilité (`scripts/recette-ha.sh`) : crash du serveur principal → bascule automatique du client sur le backup → **reprise de la transaction en cours sans double débit** → nouveau paiement traité par le backup. **13 OK / 0 KO**.

7. Gouvernance & conformité

Analyse de risques (EBIOS RM) — synthèse

Scénario	Gravité	Vraisemblance résiduelle	Niveau résiduel
DoS → indisponibilité	Critique	Faible (détecté + atténué auto)	Moyen
Injection → altération/fuite	Critique	Faible (bloqué WAF)	Faible-Moyen
Accès non autorisé	Grave	Faible	Faible

PCA / PRA

Services vitaux : paiement et authentification. **RTO** visé < 30 s (mesuré ~15 s) ; **RPO** nul pour la disponibilité (base persistante), = dernier snapshot pour une compromission. Deux playbooks distincts, responsabilités définies (automate / opérateur / équipe).

NIS2 / DORA

Le prototype démontre : gestion des risques reliée à l'architecture, détection et réponse aux incidents, continuité (reprise autonome), traçabilité. Non démontrables en 4 jours : notification réglementaire, gouvernance organisationnelle, registre des tiers TIC, TLPT.

Recommandations à la direction

- Industrialiser la reprise (haute disponibilité + WAF réseau) : meilleur rapport valeur/effort.
- Formaliser la notification d'incident à partir de la traçabilité existante.
- Planifier des tests de résilience réguliers (le dispositif d'attaque contrôlée existe déjà).
- Décider sur des éléments vérifiables : RTO, taux de blocage WAF, score d'anomalie, succès canari.

Détail : `docs/gouvernance/` dans le dépôt.

8. Contributions, limites & suites

Contribution de chacun

Membre	Domaine(s) assumé(s)	Contributions principales
[À COMPLÉTER]	Développement	API paiement, auth, frontend
[À COMPLÉTER]	Infrastructure	Compose, observabilité, déploiement
[À COMPLÉTER]	IA / Data	Détecteur ML, assistant RAG
[À COMPLÉTER]	Cybersécurité	Attaques, WAF, SOC, secrets
[À COMPLÉTER]	Gouvernance	EBIOS, PCA/PRA, NIS2/DORA

Limites assumées

- Prototype mono-hôte : pas de haute disponibilité ni de reprise multi-site.
- WAF à signatures : contournement possible par obfuscation (vs WAF réseau avec CRS).
- Bouclier DoS applicatif : protège l'application, pas une saturation réseau amont.

Ce que nous ferions différemment en contexte industriel

- WAF réseau (ModSecurity/Coraza) + blocage par IP/réputation.
- Haute disponibilité (réplication DB, plusieurs nœuds) et sauvegardes testées.
- Enrichissement du modèle (plus de features, ré-entraînement périodique).

Code, configurations, scripts d'attaque et de ML, tests et README : dépôt Git du projet (github.com/bigwihodari/Resilience-Alpha).